

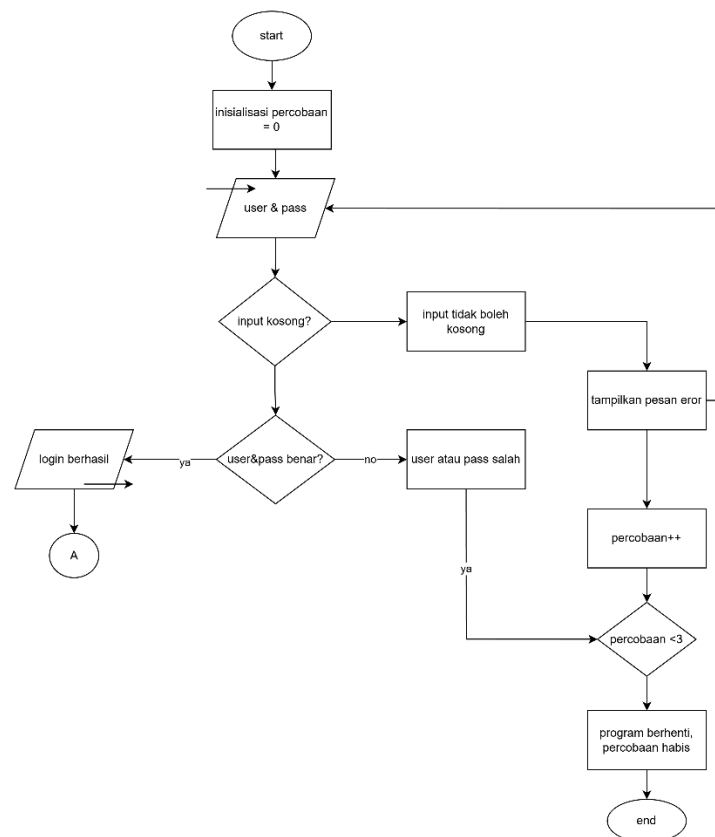
LAPORAN PRAKTIKUM
POST TEST 7
ALGORITMA PEMOGRAMAN LANJUT



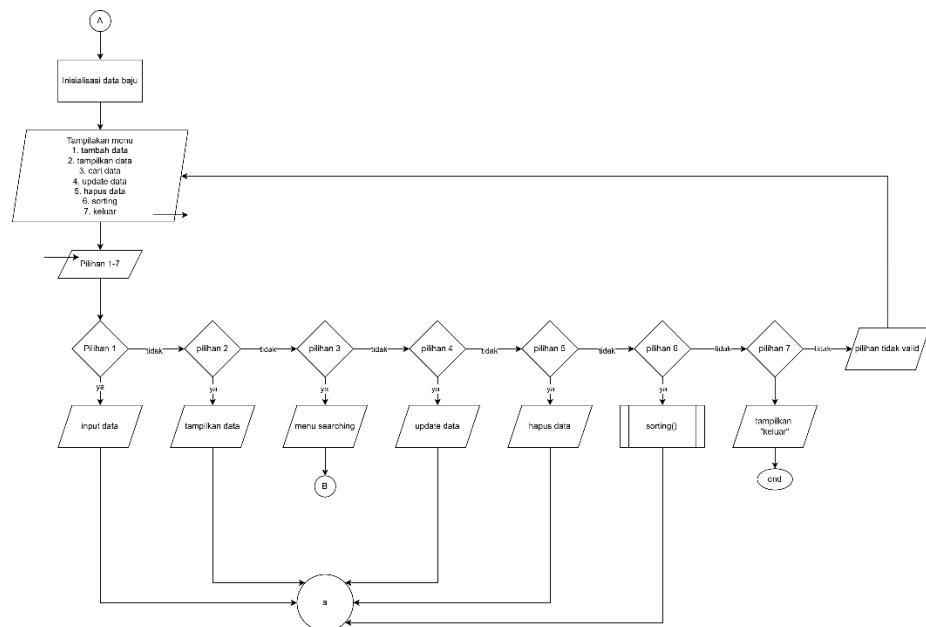
Disusun oleh:
Zihni Larasati (2509106010)
Informatika (A'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

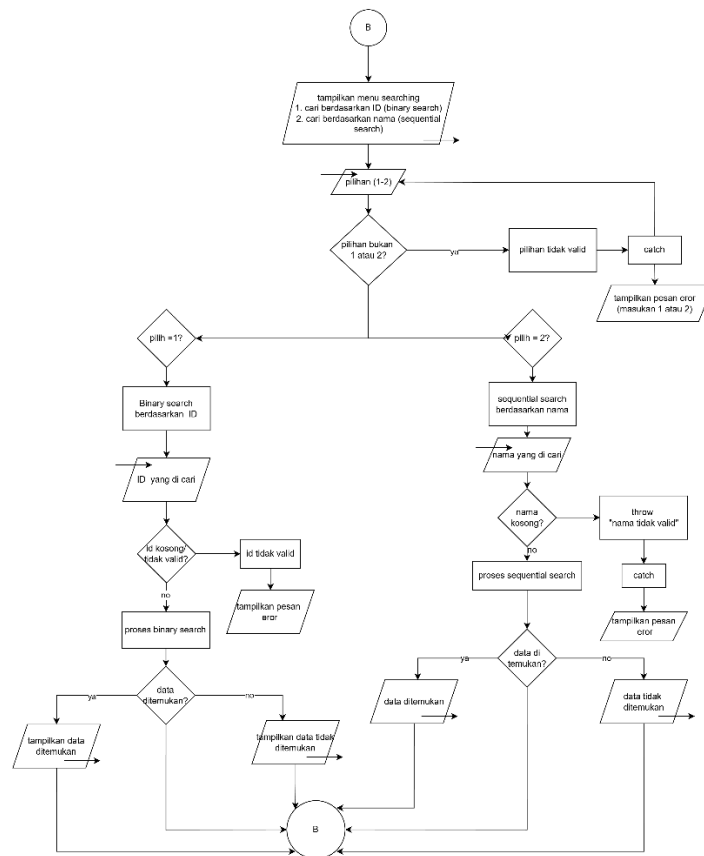
1. Flowchart



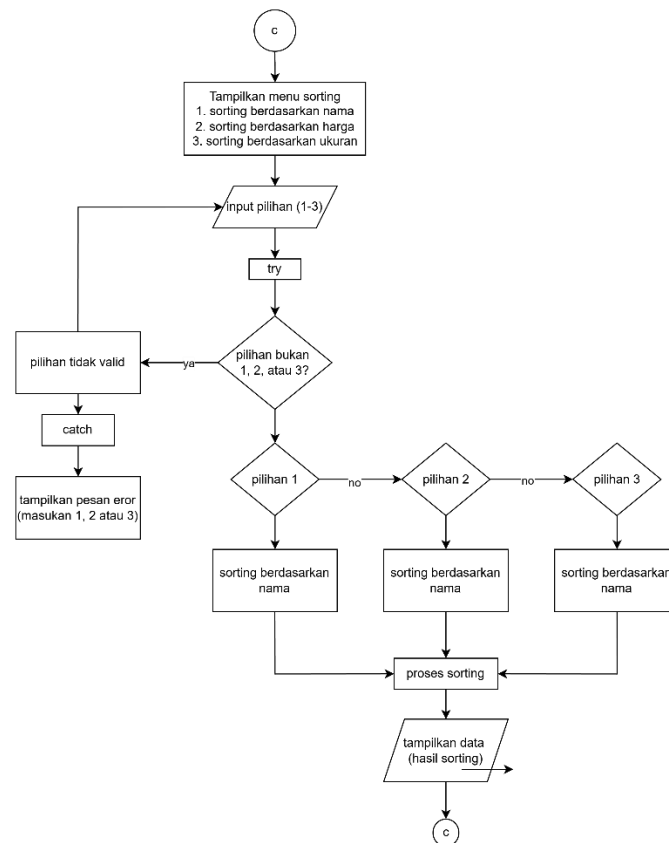
Gambar 1.1



Gambar 1.2



Gambar 1.3



Gambar 1.4

Flowchart tersebut menggambarkan alur program pengolahan data sewa baju yang diawali dengan proses login. Pengguna diminta memasukkan username dan password, kemudian sistem melakukan validasi. Jika input kosong atau salah, akan ditampilkan pesan error dan jumlah percobaan bertambah hingga maksimal tiga kali. Jika login berhasil, pengguna masuk ke menu utama yang berisi beberapa pilihan seperti tambah data, tampilkan data, cari data, update, hapus, sorting, dan keluar. Setiap pilihan akan menjalankan proses sesuai fungsinya, dan jika input tidak valid maka akan ditangani dengan error handling. Pada menu pencarian, pengguna dapat memilih metode binary search atau sequential search, sedangkan pada menu sorting, pengguna dapat memilih pengurutan berdasarkan nama, harga, atau ukuran. Secara keseluruhan, program menggunakan percabangan, perulangan, serta exception handling untuk memastikan proses berjalan dengan baik dan aman.

2. Deskripsi dan Source code

```
cout << "Harga      : ";
cout.flush();
cin >> baju[*jumlah].harga_sewa;

if(cin.fail()) {
    cin.clear();
    cin.ignore(1000, '\n');
    throw "Harga harus angka";
}
```

Gambar 2.1

```
catch(const char* err) {
    cout << "Error: " << err << endl;
}
```

Gambar 2.2

Pada bagian ini, program melakukan validasi input harga menggunakan `cin.fail()`. Jika pengguna memasukkan data selain angka, maka program akan melempar error menggunakan `throw` dan ditangkap oleh `catch`, sehingga program tidak berhenti dan menampilkan pesan kesalahan.

```
if(user == "" || pass == "")
    throw "Input tidak boleh kosong";

if(!login(user, pass))
    throw "Username / Password salah";
```

Gambar 2.3

Pada bagian login, program memvalidasi input username dan password. Jika input kosong atau salah, program akan melempar error menggunakan throw dan ditangkap oleh catch, sehingga pengguna dapat mencoba kembali tanpa menghentikan program.

```
cin >> pilih;

if(cin.fail()) throw "Input harus angka";
if(pilih < 1 || pilih > 7)
    throw "Pilihan tidak valid";
```

Gambar 2.4

Pada bagian menu, program memeriksa apakah input berupa angka dan sesuai dengan pilihan yang tersedia. Jika tidak valid, program akan melempar error dan menampilkan pesan kesalahan tanpa menghentikan program.

```
if(cin.fail()) throw "Harga harus angka";
```

Gambar 2.5

Pada bagian update, program memvalidasi input terutama pada harga. Jika input tidak sesuai (bukan angka), maka program akan melempar error dan ditangani oleh catch, sehingga proses update tetap aman.

```
throw "Data tidak ditemukan";
```

Gambar 2.6

Pada bagian hapus data, program akan mengecek apakah data dengan ID yang dimasukkan ditemukan atau tidak. Jika tidak ditemukan, program akan melempar error dan menampilkan pesan kesalahan kepada pengguna.

3. Output

```
Username: Larasati
Password: 010
Login berhasil!
```

Gambar 3.1

```
===== MENU =====
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Update Data
5. Hapus Data
6. Sorting ID
7. Keluar
```

Gambar 3.2

```
Pilih: 1

===== INPUT DATA =====
ID Baju   : 001
Nama Baju : dresspayet
Ukuran   : M
Warna     : Gold
Harga     : 500000
Status    : Tersedia
Data berhasil ditambahkan!
```

Gambar 3.3

```
Pilih: 2

=== DATA ===
1. 001 | dresspayet | M | Gold | 500000 | Tersedia
2. 002 | minidress\ | S | Hitam | 250000 | Tersedia
```

Gambar 3.4

```
Pilih: 3

1. Cari ID
2. Cari Nama
Pilih: 1
Data sudah diurutkan!
Masukkan ID: 002
Data ditemukan: minidress\
```

Gambar 3.5

```
Pilih: 3

1. Cari ID
2. Cari Nama
Pilih: 2
Masukkan Nama: minidress
Data tidak ditemukan
```

Gambar 3.6

```
Pilih: 4
Masukkan ID: 001
Data ditemukan, input ulang:
Nama Baju : dresspayet
Ukuran   : M
Warna     : Gold
Harga     : 500000
Status    : Disewa
Update berhasil!
```

Gambar 3.7

```
Pilih: 5
Masukkan ID: 002
Data berhasil dihapus!
```

Gambar 3.8

```
Pilih: 6
Data sudah diurutkan!
```

Gambar 3.9

```
Pilih: 7
Keluar program...
```

Gambar 3.10

4. GIT

```
PS C:\PRAKTIKUM_APL> git add .
PS C:\PRAKTIKUM_APL> git commit -m "Post-test-apl-7"
[main ad8240c] Post-test-apl-7
 2 files changed, 290 insertions(+)
   create mode 100644 post-test/post-test-apl-7/2509106010-ZihniLarasati-PT-7.cpp
   create mode 100644 post-test/post-test-apl-7/2509106010-ZihniLarasati-PT-7.exe
PS C:\PRAKTIKUM_APL> git push -u
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 677.42 KiB | 3.81 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Laras010/praktikum-apl
   adb6477..ad8240c  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\PRAKTIKUM_APL> █
```

Gambar 4.1